

Efficient Design Methods

Aman Arora

Logistics

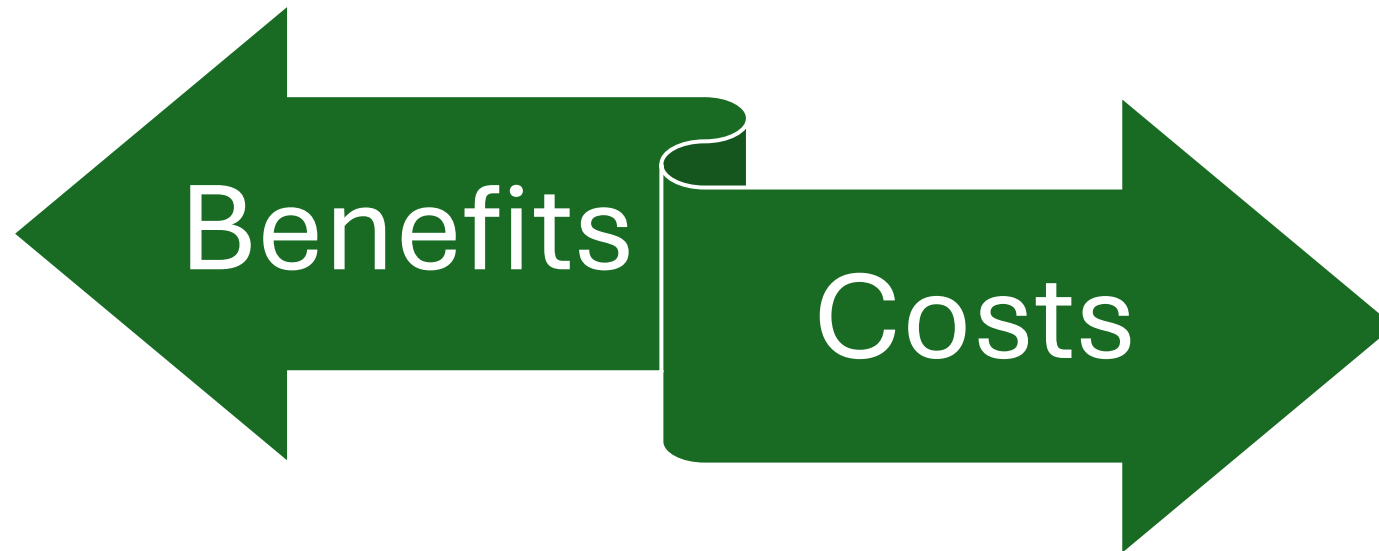
- No lecture on Wednesday
 - Friday 2-3:15pm instead
 - The lecture has been set up on the Canvas calendar

Building Blocks

- Logic/Compute
 - Arithmetic units (adders, multipliers, floating-point units)
 - Decoders, encoders
- Controllers
 - FSMs
 - Microcoded state machines
 - ASM charts
- Memory
 - FF groups
 - SRAM, Cache, Scratchpad, FIFO, RF
 - DRAM
 - CAM
- Communication
 - Buses and Network on chip
 - Point to point protocols
 - Arbiters, Routers

Tradeoffs

- There is no free lunch!
- Each optimization technique has:



Design Guidelines vs. Design Efficiency

- Modular, hierarchical design
- Abstraction (structural vs. behavioral)
- Naming conventions and coding styles

Understand what hardware are you
designing. Not what you're coding.

The Holy Grail



Power



Performance



Area

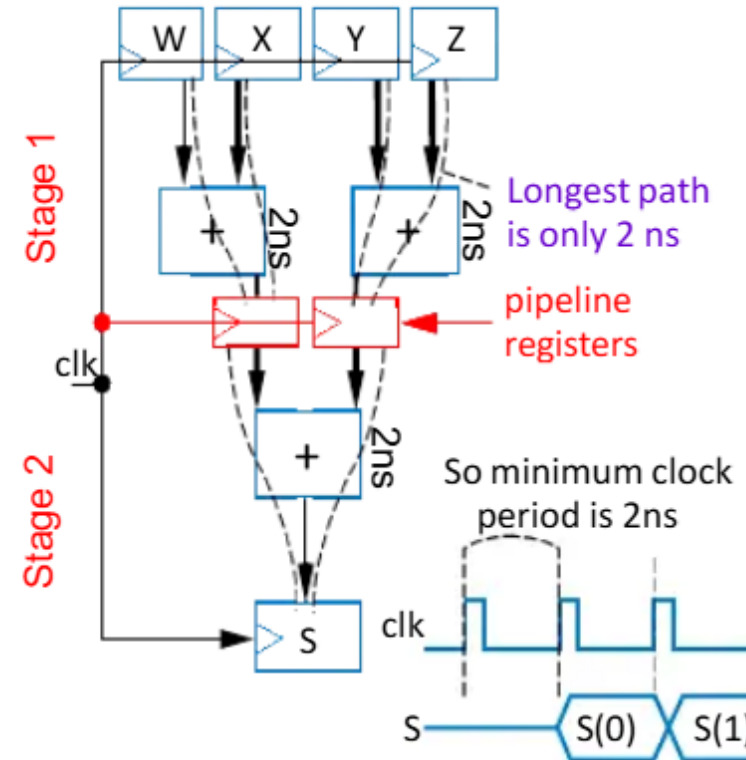
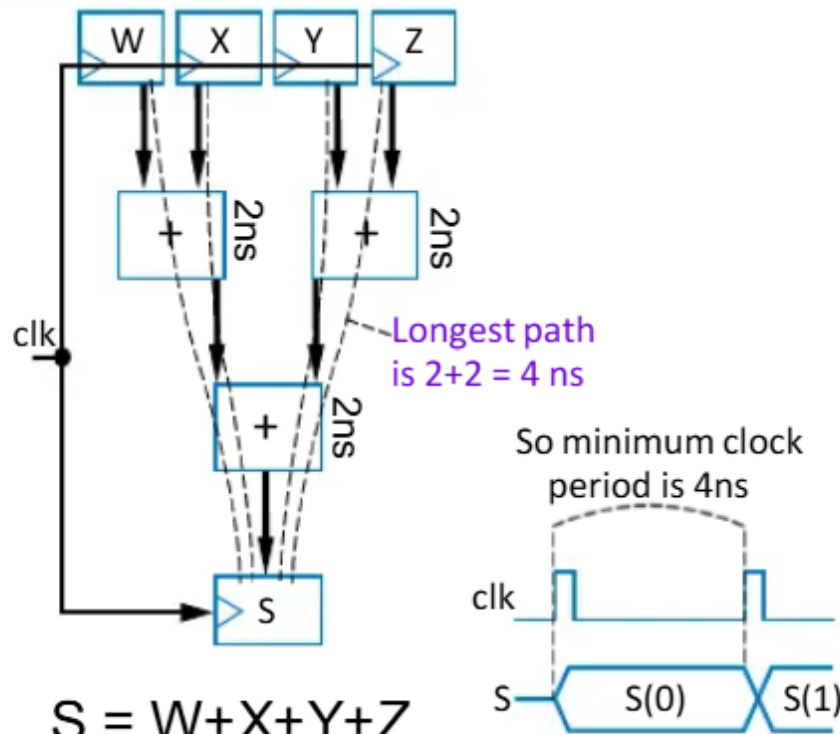
Area Optimizations

- **Resource Sharing:** Using the same hardware resources for multiple operations to reduce area.
 - E.g. explicitly instantiating a multiplier and writing control logic around it
 - E.g. writing multiplication outside of a state machine
- **Logic Minimization/Restructuring:** Simplifying and restructure logic/compute
 - E.g., Rewrite expressions to reduce the number of gates.
 - E.g., Fuse operations ($x=a+5$, $y=x+3 \rightarrow y=a+8$)
 - E.g., Instantiate optimized cells, instead of behavioral code
- **Precision/Bitwidth reduction:** Remove unnecessary bits. Use data formats that are necessary.
 - $FP32 \rightarrow INT8$, $INT16 \rightarrow INT8$

Timing Optimizations

- **Critical Path Reduction:** Identifying and optimizing the longest path in the circuit to improve overall speed.
 - E.g. reduce logic depth by logic restructuring or balanced trees
- **Pipelining:** Breaking down a process into smaller stages with registers between them to increase throughput.
- **Retiming/Rebalancing:** Adjusting the (physical/logical) position of registers to optimize timing and improve performance.
- **Logic/Register Duplication:** Reducing fanout by duplicating logic
- **Building Block Selection:** Choose adder/multiplier/etc. architecture to improve speed

Pipelining Example



- $S = W + X + Y + Z$
- Datapath on left has critical path of 4 ns, so fastest clock period is 4 ns
 - Can read new data, add, and write result to S, every 4 ns
- Datapath on right has critical path of only 2 ns
 - So can read new data every 2 ns – *doubled performance* (sort of...)

Power Optimizations

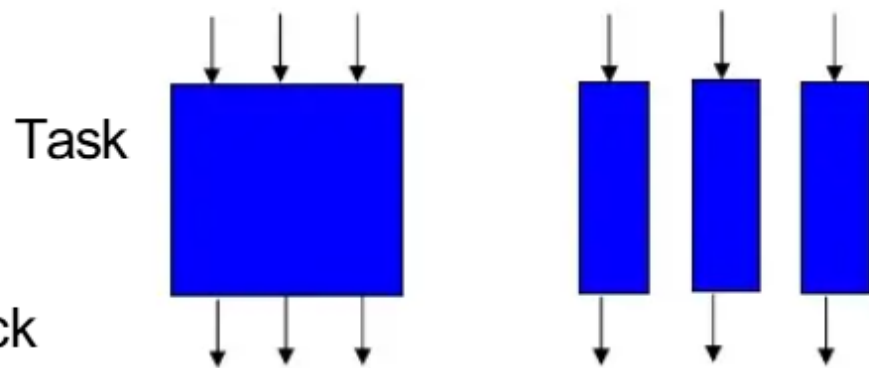
- **Clock Gating:** Reducing power consumption by disabling the clock signal to inactive parts of the circuit.
- **Power Gating:** Shutting down entire blocks when not in use to save power.
- **Operand Isolation:** Prevent unnecessary toggling inside a combinational block when its output is not needed.
- **Dynamic Voltage and Frequency Scaling (DVFS):** Adjusting voltage and frequency according to workload requirements.

More Optimizations

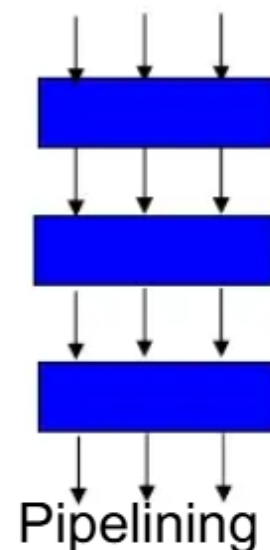
- **Concurrency:** Performing multiple operations simultaneously to speed up processing.
- **Constant Hardcoding and Propagation:** Replacing parameters with known/fixed values. Prevent over-design.
 - E.g, if you know you will every multiply by 2 or 3.
- **Precomputation with LUT replacement:** Replacing a combinational computation by a lookup table (LUT) containing its precomputed results.
- **Piecewise linear implementations:** Approximate a complicated function using several simple linear segments
 - Taylor series EXP $\rightarrow$ LUT-based EXP

Concurrency

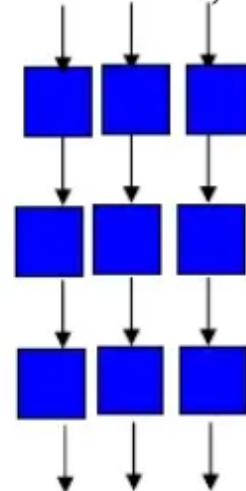
- **Concurrency**: Divide task into subparts, execute subparts simultaneously
 - Dishwashing example: Divide stack into 3 substacks, give substacks to 3 neighbors, who work simultaneously – 3 times speedup (ignoring time to move dishes to neighbors' homes)
 - Concurrency does things side-by-side; pipelining instead uses stages (like a factory line)
 - Already used concurrency in FIR filter – concurrent multiplications



a



Can do both, too



More Optimizations

- Operator strength reduction
 - Multiplication → Shifts
- Control logic simplification
 - Reduce state transitions
- Data encoding
 - Grey encoding, one-hot encoding
 - FSM state encoding to reduce switching during state transitions
- Tree restructuring/balancing
 - Adder trees, Mux trees

Synthesis and Physical Design tools can do many of these. RTL designers need to focus on what they can't.

Security Optimizations

- **Secure Boot and Encryption:** Ensuring that only authenticated firmware and data are executed or accessed (e.g. adding RSA, ECC (Elliptic Curve Cryptography), SHA, AES blocks on chips)
- **Hardware Obfuscation:** Making reverse engineering of the design more difficult (e.g., using FPGA fabric).
- **Side-Channel Attack Mitigation:** Designing to resist attacks based on power analysis, timing, or electromagnetic emissions (e.g., binary masking, dummy cycles and shuffling)
- **Access Control Mechanisms:** Restricting access to sensitive parts of the design or memory (e.g., ARM Trustzone).

Reliability/Safety Optimizations

- **Error Detection and Correction:** Implementing mechanisms like parity bits and ECC (Error-Correcting Code) to enhance reliability.
- **Redundancy Techniques:** Using hardware or logic redundancy (e.g., TMR - Triple Modular Redundancy) to tolerate faults.

Using LLMs for Design Optimizations

- [Dr. RTL: Autonomous Agentic RTL Optimization through Tool-Grounded Self-Improvement](#)
- [A New Benchmark for the Appropriate Evaluation of RTL Code Optimization](#)
- [POET: Power-Oriented Evolutionary Tuning for LLM-Based RTL PPA Optimization](#)

Exercise

Technique	Area	Delay	Power

Fill each cell with a
technique discussed
earlier

Fill each cell with “Increase” or “Decrease”

OpenTitan

- See:
 - <https://opentitan.org/book/index.html>
 - <https://github.com/lowRISC/opentitan>
- Some examples of guidelines and optimizations mentioned earlier in OpenTitan
 - Modular, hierarchical design
 - Resource sharing, reusable building blocks (hw/ip/prim)
 - Parameterization
 - Clock gating (hw/ip/prim/rtl/prim_clock_gating.sv & hw/ip/clkmgr/)
 - AES accelerator core (hw/ip/aes)
 - SECDED (hw/ip/prim/rtl/prim_secDED_*)
 - Custom FSM encodings for security (EDN, CSRNG, lifecycle controller)
 - Grey-coded pointers (FIFO (prim_fifo_async))